HLD Level 2.8

Design a Scalable Notification System — Complete Interview Question & Answer

Interview question: Design a notification platform that can send email, SMS and push notifications to millions of users. It must support transactional and bulk notifications, user preferences, priorities, retries, provider failures, rate limits, deduplication and delivery tracking.

1. Requirements

Functional: accept notification requests, support email/SMS/push, templates and personalization, user preferences, scheduling, priorities, retries, delivery status and bulk campaigns.

Non-functional: scalable throughput, low latency for urgent notifications, high availability, durable requests, no accidental duplicate sends, bounded retries, provider isolation and observability.

2. Notification Categories

```
Transactional: OTP, password reset, payment alert
Operational: device offline, system warning
Bulk/Marketing: campaigns, announcements
```

Different categories need different latency, priority and retry policies. An OTP should not sit behind a million marketing messages.

3. Capacity Example

```
10 million users
Average = 5 notifications/user/day
= 50 million/day
Average ~= 579 notifications/sec

Campaign burst: 5 million notifications in 1 hour
~= 1,389/sec before retries/multi-channel fanout
```

Average traffic can be modest while campaign or incident bursts dominate capacity planning.

4. High-Level Architecture

```
Producers / Services
|
v
Notification API
|
v
Validation + Preference/Template Resolution
|
v
Durable Message Broker
/ | \
v v v
Email Q SMS Q Push Q
| | |
Workers Workers Workers
| | |
Provider Provider Provider
\ | /
Delivery Events
|
v
Status Store
```

5. Notification API

```
POST /notifications
{
idempotency_key,
```

```
user_id,
type,
channel(s),
template_id,
template_data,
priority,
schedule_time
}
```

The API validates the request and durably accepts it; it should not synchronously wait for an external email/SMS/push provider.

6. Why a Broker?

External providers can be slow or unavailable. A durable broker decouples request acceptance from delivery, absorbs bursts, supports retries and lets channel workers scale independently.

```
API -> Broker -> Workers -> Provider
```

7. Separate Channel Queues

Email, SMS and push have different providers, costs, limits and failure modes. Separate queues/topics prevent an SMS outage from blocking email or push.

8. Priority Isolation

```
High priority: OTP / security / critical alert
Normal: transactional updates
Low: marketing/bulk
```

Prefer separate queues or weighted scheduling over one strict priority queue, because an endless stream of high-priority work can starve lower classes.

9. User Preferences

Before sending, check channel opt-in/opt-out, quiet hours, locale, destination validity and notification category. Security-critical messages may have different policy from marketing.

10. Preference Storage

```
user_id -> {
email_enabled,
sms_enabled,
push_enabled,
marketing_opt_in,
locale,
quiet_hours
}
```

Frequently read preferences can be cached, but policy correctness and cache invalidation must be defined.

11. Templates

Store versioned templates separately from delivery jobs. Workers render the chosen template version with user/event data. Versioning prevents a template edit from unpredictably changing already accepted jobs.

12. Fanout

A single logical notification may fan out to multiple channels.

```
Notification N123
+-> email job
+-> push job
+-> SMS job
```

Track each channel attempt independently while retaining the parent notification ID.

13. Idempotency

Clients may retry API calls after a timeout. An idempotency key prevents creating the same logical notification twice.

```
(tenant_id, idempotency_key) -> notification_id
```

Idempotency at request creation and deduplication at provider delivery are separate concerns; both may matter.

14. Delivery State Machine

```
ACCEPTED -> QUEUED -> SENDING -> SENT
|
+-> RETRY_WAIT -> SENDING
|
+-> FAILED / DEAD_LETTER
```

15. Provider Response Semantics

A provider returning success often means it accepted the message, not that the end user received/read it. Track states such as provider-accepted, delivered, bounced, failed or opened only when the channel/provider supports them.

16. Retries

Retry transient failures such as timeouts or provider 5xx responses using bounded exponential backoff with jitter. Do not retry permanent errors such as an invalid destination indefinitely.

```
attempt 1 -> 1 sec
attempt 2 -> 2 sec
attempt 3 -> 4 sec + jitter
... max attempts
```

17. Why Not Sleep in Worker Threads?

A worker should not sleep for minutes waiting for retry time. Put failed jobs into a delayed/scheduled retry mechanism so the worker can process other messages.

18. Dead-Letter Queue

After maximum attempts, move the job to a DLQ with error context for inspection, repair or controlled replay.

19. Duplicate Delivery

At-least-once queues can redeliver if a worker sends successfully but crashes before acknowledging the message. If the provider supports an idempotency key, pass a stable notification/attempt key. Otherwise duplicates may be impossible to eliminate perfectly and must be minimized/understood.

20. Provider Rate Limits

```
Provider allows 10,000 requests/minute
Worker capacity > provider capacity
```

Apply a token-bucket or similar limiter per provider/account so workers do not create repeated 429 responses and retry storms.

21. Multiple Providers

```
Email -> Provider A (primary)
-> Provider B (fallback)
```

Multi-provider routing improves resilience but requires explicit failover rules, cost/quality considerations and duplicate-send protection.

22. Circuit Breaker

If a provider is consistently failing, stop hammering it temporarily. Open the circuit, route to fallback where allowed, and probe recovery later.

23. Bulk Campaigns

Do not enqueue millions of personalized messages in one synchronous API request. Store a campaign definition and use a fanout service to generate jobs gradually under throughput quotas.

```
Campaign -> audience segments -> fanout workers -> channel queues
```

24. Scheduling

Scheduled notifications need a durable scheduler. Store future jobs by due time and move them into delivery queues when ready. Avoid holding millions of future timers in application memory.

25. Quiet Hours

If a user's policy says no marketing messages overnight, calculate the next eligible send time and schedule it. Time-zone handling is part of the product requirement.

26. Ordering

Global ordering is unnecessary. For some use cases, ordering per user or conversation may matter. Partition by user_id if local ordering is required, but be aware of hot users/tenants.

27. Partitioning

```
partition = hash(user_id or tenant_id) % N
```

Choose the key based on ordering and load distribution. A giant tenant can create a hot partition if tenant_id alone is used.

28. Backpressure

If provider capacity is below incoming traffic, queue depth and age increase. Bound internal memory, retain work durably in the broker, throttle bulk producers and protect high-priority traffic.

> **Queue depth tells how much work exists; oldest-message age tells how stale the user experience is. Monitor both.**

29. Overload Policy

Protect transactional/security notifications first. Slow or pause marketing fanout before allowing low-priority traffic to consume all provider quota or worker capacity.

30. Storage

Use a current-status store for notification state and an append/event/history path for attempts and provider callbacks. Long-term analytics can move to cheaper storage.

31. Provider Webhooks

```
Provider -> webhook endpoint -> validate signature -> delivery event broker -> status updater
```

Webhook processing should be idempotent because providers may retry callbacks.

32. Security and Privacy

Protect email addresses/phone numbers/device tokens, encrypt sensitive data, restrict access, validate webhook signatures, avoid logging secrets, and honor retention/deletion requirements.

33. Observability

Track accepted/sec, sent/sec by channel, queue depth, oldest queue age, worker throughput, provider latency, provider error/429 rate, retry rate, DLQ count, duplicate suppression, delivery/bounce rate and end-to-end latency by priority.

34. C++ Worker Architecture

```
Broker consumer
|
v
bounded internal queue
|
v
worker pool
|
+-> template/render
+-> preference/policy
+-> provider client
|
v
ack / retry schedule
```

Use bounded queues, condition variables for sleeping workers, connection pooling for provider HTTP clients, RAII for network/resources and short lock scopes.

35. Worker Count

Notification delivery is usually I/O-heavy, so more workers than CPU cores may be useful, but provider rate limits and connection-pool capacity matter more than blindly increasing threads.

36. Graceful Shutdown

```
mark worker not ready
stop pulling new messages
finish or safely abandon in-flight sends
persist retry/result state
ack only completed work
join workers
close clients/resources
```

Do not acknowledge a broker job before the system has reached the durability point required by the chosen delivery semantics.

37. Failure Scenario: SMS Provider Down

```
SMS provider fails
-> circuit opens
-> SMS jobs remain/retry durably
-> email/push continue normally
-> optional fallback provider
-> alerts fire
```

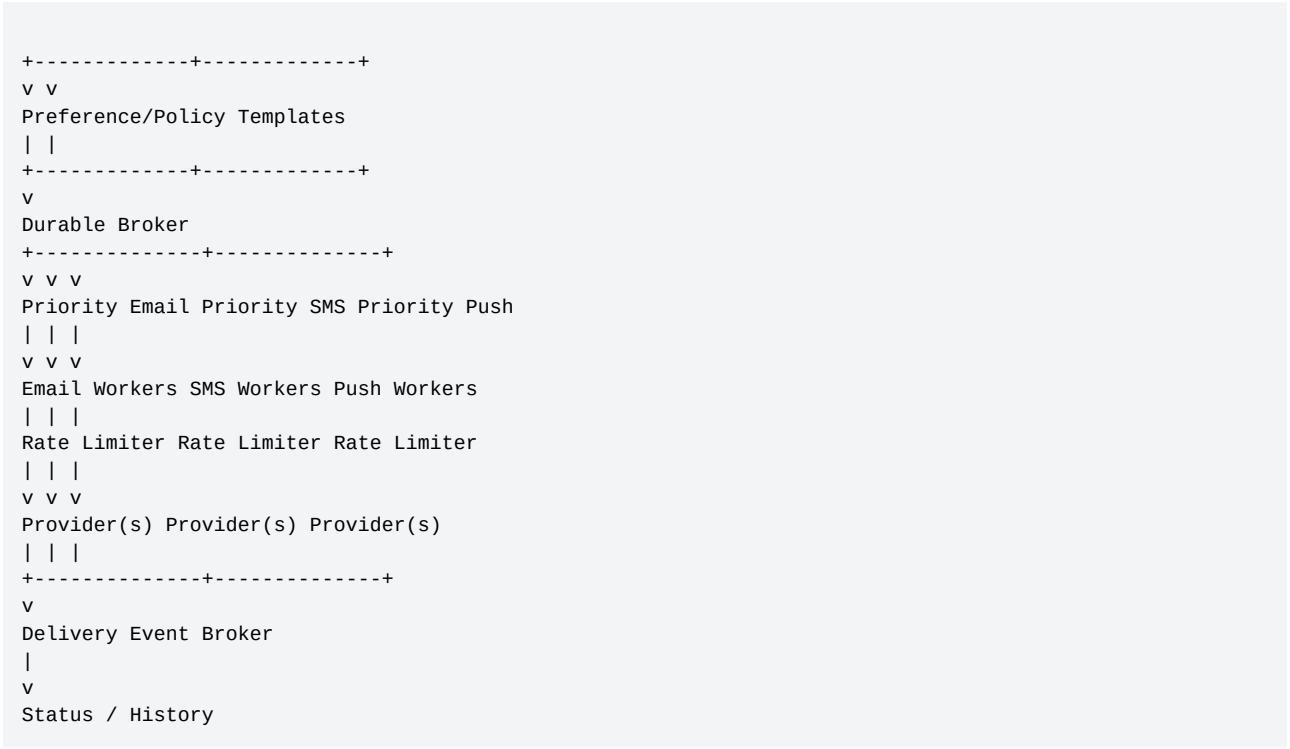
Channel isolation prevents one external dependency from becoming a full notification-platform outage.

38. Failure Scenario: Database Slow

Avoid putting nonessential synchronous database operations on every delivery path. Cache stable configuration where safe, batch status updates if appropriate, and let durable queues absorb temporary downstream slowness.

39. Complete Architecture

```
PRODUCER SERVICES
|
v
Notification API
|
```


Preference/Policy Templates
Durable Broker
Priority Email Priority SMS Priority Push
Email Workers SMS Workers Push Workers
Rate Limiter Rate Limiter Rate Limiter
Provider(s) Provider(s) Provider(s)
Delivery Event Broker
Status / History

40. Common Interview Follow-Ups

Why queues? Decouple API acceptance from slow/unreliable providers and absorb bursts.

How prevent OTP from waiting behind marketing? Priority isolation/separate queues and reserved capacity.

How prevent duplicate API submissions? Idempotency key.

What if send succeeds but worker crashes before ACK? Redelivery is possible; use provider idempotency where available and stable delivery keys.

How retry? Bounded exponential backoff + jitter using delayed scheduling, not sleeping worker threads.

What if provider is down? Circuit breaker, durable retry and optional provider failover.

How handle millions of campaign recipients? Gradual fanout with quotas, not one synchronous expansion.

What metrics matter? Queue age/depth, provider errors/latency, retry/DLQ, end-to-end latency and delivery outcomes.

41. Interview-Ready Spoken Answer

"I'd make notification submission asynchronous. Producer services call a notification API with an idempotency key, user, template, channels and priority. The API validates the request, resolves or records policy information and durably publishes work to a broker rather than waiting for an external provider. I would isolate email, SMS and push because their providers, costs and failure modes differ, and I'd isolate high-priority transactional traffic from bulk marketing so an OTP cannot sit behind a campaign. Channel workers render templates, apply user preferences, enforce provider-specific rate limits and send through pooled provider clients. Transient failures go through bounded exponential-backoff retries with jitter; permanent failures or exhausted retries go to a dead-letter queue. Because at-least-once delivery can redeliver after a worker crash, I'd use stable idempotency/deduplication keys where the provider supports them. Provider outages are contained with circuit breakers and optional fallback providers. Bulk campaigns are expanded gradually by fanout workers under quotas. I'd monitor queue depth and oldest-message age, provider latency/errors, retry and DLQ rates, and end-to-end latency by priority. In C++, workers use bounded queues, condition variables, RAII, connection pooling and graceful shutdown so unacknowledged work remains recoverable."

42. Five Decisions to Remember

1. External providers are slow/unreliable -> ASYNCHRONOUS DURABLE QUEUES.
2. OTP must not wait behind marketing -> PRIORITY/QUEUE ISOLATION.
3. Retries and broker redelivery happen -> IDEMPOTENCY + DEDUPLICATION.
4. Providers have quotas/failures -> RATE LIMITER + CIRCUIT BREAKER + OPTIONAL FAILOVER.
5. Millions of campaign recipients -> GRADUAL FANOUT + BACKPRESSURE, NOT ONE GIANT SYNCHRONOUS JOB.

43. Final Revision Flow

```
Requirements
-> channels/categories
-> capacity
-> async API + broker
-> channel/priority isolation
-> preferences/templates
-> idempotency
-> workers/provider rate limits
-> retries/DLQ
-> provider failure/circuit breaker
-> campaigns/scheduling
-> storage/webhooks
-> backpressure
-> observability
```

```
-> C++ worker deep dive
```